

SALES FORECASTING

Time-Series Forecasting Using ARIMA

Internship	Codec Technologies – 1-Month Data Analytics Internship
Project	Sales Forecasting
Prepared by	Wasif Sayyed
Domain	Retail / E-commerce Sales
Tools	Python, Pandas, NumPy, Matplotlib, Statsmodels, Scikit-learn
Model	ARIMA (1,1,1) baseline

Submission note: This report is designed as an internship portfolio project. Because no personal/company sales dataset was supplied, the accompanying script uses a clean input format and generates the actual forecast and evaluation metrics when a monthly sales CSV is provided. No fabricated sales values or performance metrics are presented.

1. Project Overview

Sales forecasting uses historical sales observations to estimate future demand. Accurate forecasts can support inventory planning, staffing, purchasing, budgeting and promotional decisions. This project builds a reproducible monthly time-series forecasting workflow using an ARIMA model.

Goal: Predict future sales based on historical monthly sales data.

Business objective: provide a forecast that can help a business plan stock, resources and future sales targets.

2. Dataset Structure

The project expects a CSV file named **monthly_sales.csv** containing at least two columns: **Date** and **Sales**. If the source contains multiple transactions per month, the script aggregates them into monthly totals.

Column	Description	Example
Date	Transaction or observation date	2025-01-01
Sales	Sales amount for the observation	12500

3. Project Methodology

1	Load data	Read Date and Sales columns and convert them to appropriate data types.
2	Clean data	Remove invalid records, duplicates and handle missing observations.
3	Aggregate	Convert transaction-level records into regular monthly sales totals.
4	EDA	Visualize historical sales, trend and rolling mean.
5	Split data	Use the first 80% for training and the last 20% for testing.
6	Train ARIMA	Fit an ARIMA(1,1,1) baseline model on training sales.
7	Evaluate	Calculate MAE, RMSE and MAPE on the held-out test period.
8	Forecast	Refit on all available data and forecast the next six months.

4. Exploratory Data Analysis

The first chart shows the historical monthly sales series. A rolling mean is also plotted to make the underlying trend easier to see. The analyst should look for upward/downward trends, sudden shocks, outliers, long periods of growth or decline and possible seasonal patterns.

- Trend: whether average sales are increasing or decreasing over time.
- Seasonality: repeated monthly or quarterly patterns that may affect demand.
- Outliers: unusual spikes or drops caused by promotions, stockouts, holidays or one-time events.
- Data gaps: missing months should be investigated before forecasting.

5. ARIMA Model

ARIMA stands for AutoRegressive Integrated Moving Average. It models time-series behavior using autoregression, differencing and moving-average components. The baseline used here is ARIMA(1,1,1): **p=1** uses one autoregressive lag, **d=1** applies first-order differencing, and **q=1** uses one moving-average term.

For a production system, the order should be tuned using time-series validation, AIC/BIC, residual diagnostics and domain knowledge. If strong seasonality exists, a seasonal model such as SARIMA should be considered.

6. Train/Test Strategy

Time-series data must not be randomly shuffled because that would allow future information to influence the past. This project therefore keeps chronological order: the first 80% of observations are used for training and the final 20% are held out for testing.

7. Evaluation Metrics

Metric	Meaning
MAE	Average absolute difference between actual and predicted sales.
RMSE	Square-root of mean squared error; penalizes larger errors more strongly.
MAPE	Average absolute percentage error; useful for relative forecast error when actual sales are non-zero.

8. Results Table

The final MAE, RMSE and MAPE depend on the actual sales dataset. The script prints these values after execution. This report intentionally does not invent results.

Metric	Value
MAE	Generated after running the script
RMSE	Generated after running the script
MAPE	Generated after running the script

9. Complete Python Implementation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("monthly_sales.csv")
df["Date"] = pd.to_datetime(df["Date"])
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")

df = df.dropna(subset=["Date", "Sales"]).drop_duplicates()
df = df.sort_values("Date")

df["Month"] = df["Date"].dt.to_period("M")
monthly = df.groupby("Month")["Sales"].sum().to_timestamp()
monthly = monthly.asfreq("MS").interpolate(method="linear")

# Historical sales
plt.figure(figsize=(10, 5))
plt.plot(monthly.index, monthly.values)
plt.title("Historical Monthly Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()

# Chronological train/test split
split = int(len(monthly) * 0.80)
train = monthly.iloc[:split]
test = monthly.iloc[split:]

# ARIMA baseline
model = ARIMA(train, order=(1, 1, 1))
fitted = model.fit()
```

```

forecast = fitted.forecast(steps=len(test))
forecast.index = test.index

mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mean_squared_error(test, forecast))

denominator = test.replace(0, np.nan)
mape = (np.abs((test - forecast) / denominator).dropna()).mean() * 100

print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("MAPE:", round(mape, 2), "%")

# Actual vs forecast
plt.figure(figsize=(11, 5))
plt.plot(train.index, train.values, label="Train")
plt.plot(test.index, test.values, label="Actual Test")
plt.plot(forecast.index, forecast.values, label="Forecast")
plt.title("Actual vs Forecast Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.tight_layout()
plt.show()

# Next six months
final_model = ARIMA(monthly, order=(1, 1, 1)).fit()
future = final_model.forecast(steps=6)

future_dates = pd.date_range(
    monthly.index[-1] + pd.offsets.MonthBegin(1),
    periods=6,
    freq="MS"
)
future.index = future_dates

future_table = pd.DataFrame({
    "Date": future.index,
    "ForecastSales": future.values
})

print(future_table)
future_table.to_csv("future_sales_forecast.csv", index=False)

```

10. Business Applications

- **Inventory planning:** estimate future demand to reduce overstock and stockout risk.
- **Staffing:** prepare staffing levels around expected high/low sales periods.
- **Purchasing:** support procurement decisions using expected demand.
- **Budgeting:** create more informed revenue and cash-flow expectations.
- **Marketing:** compare forecasted demand with promotional plans and investigate unusual deviations.

11. Seasonal & Promotional Extensions

The internship guideline asks analysts to consider seasonal, promotional and trend components. The baseline ARIMA project can be extended by adding promotion indicators and using SARIMA for recurring seasonality. Prophet or LSTM can also be evaluated as alternative models. The important point is to compare models using a chronological validation strategy rather than selecting a model only because it fits the training data.

12. Limitations

- The baseline requires a sufficiently long and regular time series.
- ARIMA(1,1,1) is a baseline and should not automatically be treated as the best possible model.
- External drivers such as price, promotions, holidays, competitors and stock availability are not included unless added as features.
- Forecast quality depends strongly on the quality and granularity of the underlying sales data.

13. Conclusion

This project demonstrates an end-to-end sales forecasting workflow: data preparation, monthly aggregation, time-series visualization, chronological train/test splitting, ARIMA modeling, error evaluation and future forecasting. It provides a strong portfolio foundation that can be extended with SARIMA, Prophet, LSTM, holiday variables and promotional data.

14. GitHub Submission Checklist

- monthly_sales.csv (or a README explaining the dataset source)
- sales_forecasting.py
- Sales_Forecasting_Internship_Project.pdf
- future_sales_forecast.csv (generated after running the script)
- README.md with objective, dataset, methodology, results and business insights

Prepared for Codec Technologies Data Analytics Internship. For educational/portfolio use.